

INFORME TÉCNICO Y DE RETROSPECTIVA: "KNOW YOUR MARKS"

Proyecto: Suite interactiva de benchmarking, planeador de componentes de hardware y registro de diagnósticos.

Autores:

Tecnologías Principales: Vue 3 (Frontend) + Django REST Framework (Backend) + PostgreSQL (Persistencia)

Aspectos técnicos

1. Arquitectura y Estructura General del Proyecto

Para el desarrollo de "Know Your Marks", se implementó una arquitectura desacoplada (cliente-servidor) utilizando el patrón de diseño de múltiples capas. Esto permite que el Frontend y el Backend funcionen como entidades independientes que se comunican de manera segura a través de un protocolo común (HTTP/JSON).

A. Capa de Presentación (Frontend)

Ubicada en la carpeta /project-frontend, está construida como una Single Page Application (SPA) moderna:

- Framework: Vue 3 utilizando la Composition API (<script setup lang="ts">), lo que nos permitió manejar un estado reactivo y limpio.
- Modularización de Vistas: La interfaz se dividió en componentes altamente reutilizables y especializados para mantener el código legible.
- App.vue: Orquestador principal del estado global, simulación de benchmarking y control de la sesión activa del laboratorio.
- OverviewPage.vue: Panel principal que renderiza gráficos vectoriales dinámicos (estabilidad de voltajes y frecuencias) mediante SVG y contadores de rendimiento en tiempo real (FPS, temperaturas).
- HardwareLibraryPage.vue: Catálogo interactivo de recomendaciones de mejoras calculadas según el hardware actual.
- MyHardwarePage.vue: Gestor de especificaciones técnicas del usuario, con calculadora interactiva de consumo de potencia (TDP) y análisis de riesgo térmico.
- AdminConsolePage.vue: Panel administrativo con operaciones CRUD para agregar, actualizar o eliminar componentes del catálogo, además de herramientas de exportación de reportes técnicos (CSV/PDF) y visor de cargas JSON de la API.
- Header.vue & Sidebar.vue: Elementos de navegación y utilidades del sistema.
- LoginScreen.vue: Portal de acceso y validación de credenciales para iniciar sesión.
- Estilos: Tailwind CSS v4, que elimina el exceso de archivos de hojas de estilo individuales y unifica el diseño mediante clases de utilidad y tipografías personalizadas (Space Grotesk e Inter).

B. Capa de Lógica de Negocio (Backend)

Ubicada en la carpeta /project-backend, actúa como el cerebro del sistema:

- Tecnología: Django junto con Django REST Framework (DRF).
- Controladores y Endpoints: Se estructuraron rutas RESTful en urls.py para exponer los datos del catálogo en formato JSON (por ejemplo: /api/components y /api/telemetry).
- Lógica de Intercambio: Se encarga de procesar las peticiones del frontend, validar que los datos cumplan con las reglas de negocio (como años de lanzamiento válidos y cálculos lógicos de puntajes base) y formatear las respuestas de manera segura.

C. Capa de Persistencia de Datos (Base de Datos)

- Tecnología: PostgreSQL, un motor de bases de datos relacional.
- ORM (Object-Relational Mapping): Se utilizó el ORM nativo de Django, lo que nos permitió interactuar con la base de datos utilizando objetos y código Python en lugar de escribir consultas SQL manuales, esto reduce la probabilidad de inyección de código SQL y facilita la manipulación de datos.
- Migraciones: El historial de cambios en la estructura de los modelos se gestionó a través del sistema de migraciones automáticas de Django (makemigrations y migrate), garantizando la consistencia de las tablas tanto en el desarrollo local como en producción.

2. Desafíos Técnicos, Autoaprendizaje y Lecciones Aprendidas

Como estudiantes, la transición de hacer programas pequeños en consola o proyectos locales simples a construir una aplicación full-stack conectada representó un desafío en cuanto a conocimientos y aplicación.

A. ¿Qué costó más en el proceso de autoaprendizaje con respecto a herramientas y lenguajes?

La Reactividad en el Frontend (Vue 3):

Entender que en Vue el estado de una variable (ref, computed) cambia la interfaz de forma reactiva y automática sin que tengamos que hacer un refresco manual en el DOM fue un choque conceptual, pero útil una vez dominado.

Entender el Flujo Cliente-Servidor Desacoplado:

Comprender que el Frontend (ejecutándose en el navegador del usuario) y el Backend (ejecutándose en un servidor Python independiente) son dos mundos separados que solo se comunican mediante cadenas de texto formateadas en JSON, tuvimos que entender cómo estructurar peticiones asíncronas y cómo mapear la respuesta JSON cruda del backend para transformarla en componentes visuales interactivos.

Configuración de Entornos Locales y Dependencias:

Configurar bases de datos PostgreSQL, instalar controladores como psycopg2, lidiar con versiones de Python incompatibles, y configurar entornos virtuales (venv) nos costó horas de resolución de conflictos, fue la primera vez que nos enfrentamos a herramientas de infraestructura como Docker Compose (setup.sh/setup.bat) para facilitar la inicialización automática de la base de datos.

B. Desafíos específicos del Backend y soluciones adoptadas

El dolor de cabeza de CORS (Cross-Origin Resource Sharing):

El problema intentaba que el frontend (en el puerto 5173) consumiera la API de Django (en el puerto 8000), el navegador bloqueaba las llamadas inmediatamente por políticas de seguridad de origen cruzado, no entendíamos por qué la consola se llenaba de errores rojos si el código del servidor estaba perfecto.

La solución: Aprendimos que en desarrollo es mucho más limpio configurar un Proxy en el servidor de Vite (vite.config.ts), este proxy desvía las rutas /api de forma transparente hacia el backend en segundo plano, haciéndole creer al navegador que todo proviene del mismo origen y evitando tener que relajar excesivamente las políticas de seguridad en Django.

Conflictos con las Migraciones de Django:

El problema era que modificábamos un atributo de un modelo en Python, corríamos el servidor y todo fallaba porque la base de datos PostgreSQL no se había enterado del cambio, en la desesperación, a veces borrábamos carpetas enteras de migraciones, rompiendo la sincronización del historial.

La solución: Comprendimos el flujo estricto del ORM donde cada modificación de modelo requiere generar un archivo de migración intermedio (`python manage.py makemigrations`) y luego aplicarlo formalmente (`python manage.py migrate`), aprendimos que la base de datos tiene un orden que debe respetarse.

El Control de Variables de Entorno y Credenciales:

El problema es que inicialmente tendíamos a dejar las contraseñas de la base de datos y la clave secreta de Django escritas directamente en el código de configuración.

La solución: Adoptamos el uso de archivos `.env` (excluidos en `.gitignore`), leyendo las variables con bibliotecas como `python-dotenv`, así aprendimos las buenas prácticas de seguridad de la industria.

C. Errores que con seguridad NO volveremos a cometer

Subir entornos virtuales (`venv/`) o módulos de Node (`node_modules/`) a GitHub:

En nuestros primeros proyectos subíamos carpetas de gigabytes de peso a los repositorios. Ahora entendemos la importancia crucial de configurar correctamente un archivo `.gitignore` y que los proyectos deben compartirse ligeros, confiando en archivos de dependencias como `package.json` o `requirements.txt`.

Desarrollar código sin mirar la consola de herramientas de desarrollo (F12):

Antes nos frustrábamos cuando un botón no hacía nada e intentábamos adivinar el error cambiando líneas de código al azar, ahora, nuestra primera acción es abrir la consola del navegador y el terminal del backend para leer el código de estado HTTP (200, 400, 404, 500) y los logs de error específicos.

No usar Entornos Virtuales en Python:

Instalar todas las librerías globalmente en el sistema operativo solía romper otros proyectos de la universidad debido a conflictos de versiones, hoy en día, la creación de un entorno virtual (python -m venv venv) es nuestro primer paso obligatorio al iniciar cualquier backend.

Modificar la estructura de datos sobre la marcha sin planificar:

Escribir modelos en Django sin antes haber dibujado un diagrama entidad-relación claro nos costaba tener que reconstruir la base de datos desde cero constantemente, aprendimos que 15 minutos de planificación en papel ahorran 3 horas de depuración de migraciones corruptas.

Reflexión final

Aprendizajes obtenidos durante el desarrollo:

Durante el desarrollo de este proyecto, aprendimos la importancia de saber distribuir bien el trabajo, con tiempo y teniendo en cuenta las habilidades y los conocimientos de cada uno, a través de las metodologías y patrones aprendidos en clase, como en nuestro caso utilizamos kanban, sentimos que el hecho de no tener fechas específicas para entregar los avances hizo que tuviéramos cierto desorden a la hora de ir revisando qué hacía falta hasta la fecha para los entregables y tareas asignadas

Temas que consideramos no abordados o que sería valioso profundizar en futuras ediciones del curso

Siendo sinceros el curso se disfrutó mucho a pesar de todos los inconvenientes por paro y festivos, estamos satisfechos con lo abordado debido a que al ser la mayoría práctico nos permitía estar más involucrados con nuestro aprendizaje, el uso de diapositivas con textos breves, imágenes y diferentes colores hacía que fuera más entretenido y llamativo verlo, en general el curso se hizo muy ameno tanto por el contenido de la clase como por la forma en la que se presentaron los temas.

Problemas, limitaciones o restricciones que hayan dificultado el proceso de aprendizaje.

Tuvimos inconvenientes con la comunicación interna en distintas etapas del desarrollo del proyecto, además de los inconvenientes provocados por el paro como la ausencia de uno de nuestros compañeros para las etapas finales del proyecto, haciendo que la organización de cómo se repartieron las cosas se complicara, en general sentimos que la mayor dificultad que se tuvo este semestre fue el paro y las vacaciones realizadas que cortaron nuestra continuidad en el proyecto y el ritmo de aprendizaje